Automated Root Cause Analysis Using a LangChain-Based RAG Anomaly Explanation Agent

Abstract

This paper presents an AI-based Root Cause Analysis (RCA) agent for MLOps monitoring environments. The proposed system automates the initial stage of incident investigation by responding to monitoring alerts, collecting supporting evidence from metrics, drift indicators, and dashboard observations, retrieving analogous historical incidents through a LangChain-based Retrieval-Augmented Generation (RAG) pipeline with Gemini embeddings and FAISS, and generating a structured RCA report using Gemini. The resulting report is stored as a Markdown file and delivered as a formatted HTML email, thereby reducing manual effort, improving reporting consistency, and enabling faster operational decision-making.

The workflow is initiated when Alertmanager sends an alert webhook to a FastAPI-based RCA service. The service orchestrates evidence collection, historical incident retrieval, and report generation in a fully automated sequence. Although the current prototype is demonstrated on a demand-type recall-drop scenario, the architecture is readily extensible to other monitoring alerts, incident categories, and model operations use cases.

Index Terms— MLOps, Root Cause Analysis, Retrieval-Augmented Generation, LangChain, FAISS, Gemini, Prometheus, Alertmanager, FastAPI.

1. Introduction

Machine Learning systems deployed in production require continuous monitoring to detect performance degradation, data quality issues, and service instability. Although monitoring platforms can identify abnormal conditions, they do not explain the underlying cause or indicate the most appropriate corrective action. In practice, root cause analysis (RCA) remains largely manual and typically involves inspecting dashboards, reviewing logs, consulting historical incidents, and preparing summary reports.

This work addresses that gap by introducing an AI-driven RCA agent for MLOps environments. Upon receiving a monitoring alert, the system automatically gathers supporting evidence, retrieves relevant historical incidents using Retrieval-Augmented Generation (RAG), and produces a structured RCA report. The main contribution of this work is the integration of monitoring, retrieval, and generative AI into a unified operational workflow that reduces investigation time, improves analytical consistency, and supports faster and more informed decision-making.

In this context, a **demand type** refers to an issue or service category associated with vehicle-related leads, such as battery-related, tyre-related, or other maintenance-oriented cases, that is used for monitoring and downstream analysis. The RCA prototype is demonstrated on a demand-

type recall-drop scenario because the underlying BMW MLOps system already organizes leads, monitoring outputs, and downstream reporting around these categories.

2. Related Work

Root cause analysis in production ML systems traditionally depends on monitoring platforms, dashboard inspection, and manual knowledge sharing among engineers. Recent progress in Retrieval-Augmented Generation (RAG) and large language models has enabled AI systems to augment operational workflows by combining structured evidence with historical context. However, many practical implementations remain focused on general-purpose assistants or incident summarization rather than fully integrated, alert-triggered RCA workflows for MLOps environments. The present work contributes by combining monitoring alerts, evidence collection, vector retrieval, and report generation in one automated pipeline tailored to model operations.

3. Proposed System Overview

The proposed system unifies monitoring, alerting, evidence collection, retrieval, and generative AI into a single RCA workflow. Prometheus continuously monitors model and service metrics, Alertmanager forwards alert webhooks, FastAPI coordinates the RCA stages, the retrieval layer identifies relevant historical incidents using Gemini embeddings and FAISS, and Gemini generates the final RCA report. This design transforms a fragmented and manual investigation process into a structured and repeatable workflow that produces an actionable report for immediate operational use.

4. Proposed Methodology

Evidence Collection: The RCA service first gathers all evidence relevant to the alert. This includes current metric values, feature drift indicators, dashboard observations, and alert metadata such as alert name, demand type, and severity. These inputs are consolidated into a structured evidence package that serves as the factual basis for subsequent retrieval and report generation.

RAG Retrieval: Historical incidents are stored as text documents and transformed into vector embeddings using the Gemini embedding model. These embeddings are indexed in FAISS to support efficient similarity search over prior incident records. When a new incident occurs, the system retrieves the most contextually relevant cases and uses them as supporting knowledge for downstream report generation.

Report Generation: The report generator combines the evidence package with the retrieved historical context to construct a structured prompt for the Gemini language model. Based on this input, the model produces an RCA report that includes the alert summary, likely root cause, supporting evidence, historical comparison, recommended actions, decision recommendation, confidence level, and prevention guidance. Because the output is grounded in both structured evidence and retrieved incident context, it serves not only as a technical explanation but also as an operational artifact that can be reviewed, shared, and acted upon by stakeholders.

A key design choice in the methodology is that the generated RCA report is grounded in explicit evidence rather than produced from the alert text alone. The prompt construction stage therefore combines structured numerical evidence, retrieved historical narratives, and alert metadata in a controlled format. This grounding strategy improves interpretability, reduces unsupported explanation patterns, and makes the generated report more useful in operational review settings.

5. Similarity Measure for Historical Incident Retrieval

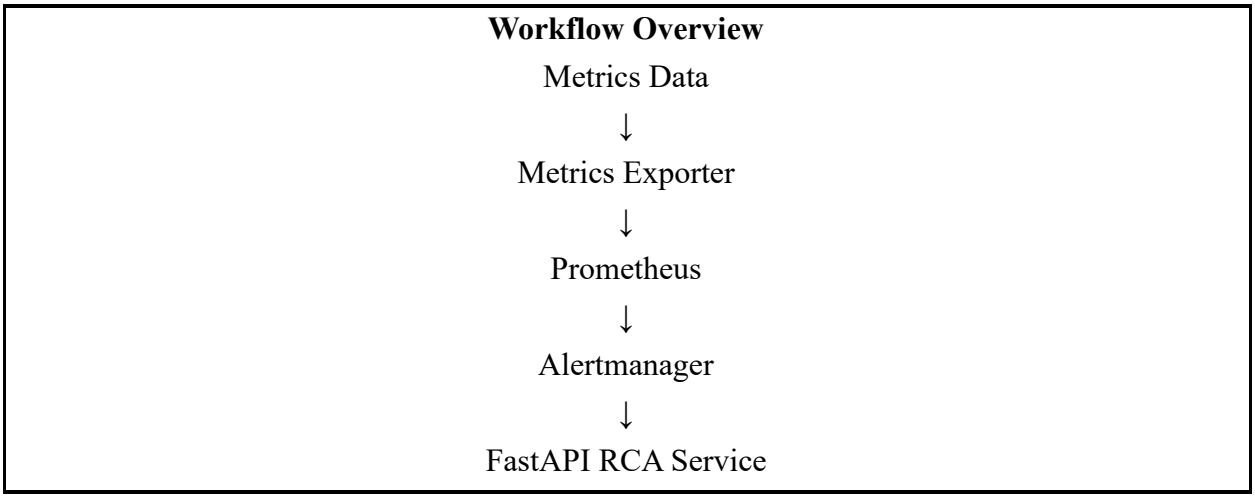
In the current prototype, historical incident retrieval is implemented using Gemini embeddings and a FAISS vector index. When a new incident query is constructed, it is embedded and compared with previously indexed incident embeddings. The retriever is configured to return the top **2** most similar incident chunks as contextual support for report generation. A common way to describe similarity between a query embedding **q** and a document embedding **d** is cosine similarity, defined as follows:

$$\textbf{CosineSimilarity}(q,d) = (q \cdot d) / (||q||\ ||d||)$$

In this expression, **q** represents the embedding of the current incident and **d** represents the embedding of a historical incident. A higher cosine similarity value indicates stronger contextual similarity between the two incidents. In the implemented system, the retriever returns the top **2** most similar results, and those retrieved incidents are inserted into the prompt sent to the report generator.

6. System Architecture and Workflow

The system architecture is organized as an alert-driven pipeline that links monitoring, evidence aggregation, retrieval, and AI-based report generation into one continuous workflow. Prometheus continuously monitors exposed model and service metrics, Alertmanager routes significant alert events to the FastAPI RCA service, and the RCA service coordinates evidence collection, retrieval of analogous historical incidents, report generation, storage, and delivery. This modular architecture separates monitoring concerns from reasoning and reporting logic, which improves maintainability and makes the system easier to extend to additional alert types or models.



↓

Evidence Collection

↓

RAG Retrieval

↓

Gemini Report Generation

↓

Report Storage and Email Delivery

7. System Components

TABLE I

MAJOR COMPONENTS OF THE PROPOSED SYSTEM

Component	**Technology**	**Purpose**
Monitoring	Prometheus	Collects model and service performance metrics.
Alerting	Alertmanager	Triggers and routes alerts to the RCA service.
Dashboard	Grafana	Visualizes real-time system and model health.
API Layer	FastAPI	Orchestrates the end-to-end RCA pipeline.
Retrieval	LangChain + FAISS	Finds and ranks similar historical incidents.
AI Model	Google Gemini (LLM)	Generates embeddings and the final RCA report.
Email Delivery	Gmail SMTP (App Password)	Sends the RCA report to the intended recipient.

These components operate as an interconnected stack rather than as isolated services. Monitoring and alerting provide the trigger, the API layer coordinates execution, the retrieval layer injects historical memory into the process, and the language model converts both structured and unstructured evidence into a usable RCA narrative. Together, they form a pipeline that combines operational telemetry with AI-assisted reasoning.

8. Implementation Steps

To operationalize the proposed RCA framework, the system is implemented as a sequence of interconnected components that collectively support monitoring, alert handling, retrieval, and report generation. The key implementation steps are summarized below.

1. Establish a structured project layout with dedicated folders for application logic, monitoring configurations, data inputs, generated outputs, and the retrieval index.
2. Configure environment variables for the Gemini API key, email sender credentials, Gmail App Password, and recipient address.

3. Implement a metrics exporter that reads monitoring values from a structured data source and exposes them through a Prometheus-compatible HTTP endpoint.
4. Deploy Prometheus, Alertmanager, and Grafana through Docker Compose as a coordinated monitoring and visualization stack.
5. Configure Prometheus to scrape the exporter at fixed intervals and define alert rules for threshold-based model degradation.
6. Configure Alertmanager to forward firing alerts as webhooks to the FastAPI-based RCA service.
7. Develop the FastAPI RCA service to orchestrate evidence collection, historical retrieval, report generation, report storage, and email delivery.
8. Implement the evidence collection module to consolidate metrics, drift indicators, dashboard observations, and alert metadata into a unified evidence package.
9. Build the retrieval layer by embedding historical incident records with Gemini, storing them in FAISS, and retrieving the most relevant incidents for a new alert.
10. Implement the report generation and output layer so that the RCA report is produced in structured form, saved to disk, and delivered as a formatted HTML email.

9. Deployment and Runtime Flow

At runtime, the monitoring and visualization stack is deployed using Docker Compose, while the FastAPI RCA service and metrics exporter run as Python services. Prometheus periodically scrapes the metrics endpoint, evaluates alert rules, and forwards firing alerts to Alertmanager. Alertmanager then triggers the FastAPI RCA service through a webhook, which initiates evidence collection, historical retrieval, report generation, report storage, and email delivery. This deployment pattern keeps the monitoring stack modular while allowing the RCA service to remain flexible for further extension or integration with external systems.

From an execution perspective, the workflow can be viewed as a coordinated multi-service runtime. One process exposes metrics, another hosts the FastAPI RCA service, while the Dockerized monitoring stack continuously evaluates system state in parallel. This separation improves modularity and mirrors how monitoring, orchestration, and AI-assisted analysis may be deployed as loosely coupled services in larger production environments.

10. Practical Example of Alert Processing

To illustrate the end-to-end behavior of the proposed system, consider a demand-type incident in which recall degrades below the operational threshold. Prometheus detects the degradation through the exported metric stream, the alert is routed by Alertmanager, and the FastAPI RCA service is invoked automatically. The service then gathers the latest evidence, including current metric values, drift signals, and dashboard observations, before retrieving the most relevant historical cases from the incident repository.

If the retrieved historical incident shows a pattern of data-quality degradation rather than model obsolescence, the generated RCA report may recommend fixing the feature pipeline instead of

retraining the model. This distinction can help guide engineering effort away from unnecessary retraining and toward the likely source of degradation. In this way, the system does not merely summarize an alert, but supports a more targeted and defensible corrective action.

TABLE II
ILLUSTRATIVE EXAMPLE OF ALERT PROCESSING

Stage	**Illustrative Output**
Alert Detection	Recall falls below threshold for a monitored demand type.
Evidence Collection	Metrics, drift indicators, and dashboard observations are gathered.
Historical Retrieval	Prior incident with similar data-quality symptoms is retrieved.
AI Report Generation	The RCA report may recommend fixing the feature pipeline instead of immediate retraining.
Operational Outcome	Stakeholders receive a structured RCA report with recommended action.

11. Results and Discussion

TABLE III
ILLUSTRATIVE COMPARISON OF MANUAL AND AI-BASED RCA PROCESSES

Metric	**Manual RCA**	**AI-Based RCA**
Investigation Time	Hours	Minutes
Reporting Consistency	Variable	High
Dependence on Human Expertise	High	Lower
Use of Historical Knowledge	Manual Search	Automatic Retrieval

As illustrated in Table III, the proposed approach is intended to improve both the speed and consistency of incident analysis. The system consolidates evidence from multiple sources, including metrics, dashboard observations, and historical incidents, into a single structured output within minutes, whereas a manual RCA process may require hours. By automating evidence gathering and report drafting, the AI agent is intended to support faster operational response, improve reporting repeatability, and reduce dependence on individual expert knowledge through systematic reuse of prior incident context.

A representative example involves an alert triggered by a sudden drop in recall for a specific demand type. In this scenario, the evidence package indicates stable API health but a sharp

increase in a feature null-rate, while the retrieval layer surfaces a prior incident with a comparable data-quality pattern. The generated RCA report may therefore recommend fixing the data pipeline rather than retraining the model, illustrating how the system can support more targeted operational decisions by combining current evidence with historical analogies.

From a broader operational perspective, the system's value extends beyond faster triage. Because the output is structured and consistently formatted, it can also improve communication between engineering teams, on-call responders, and non-technical stakeholders who require concise summaries of incident causes and actions. In this sense, the RCA report serves not only as an explanation artifact but also as an organizational coordination tool.

12. Limitations

Despite its benefits, the current prototype has several limitations. It relies on simulated, static data sources rather than a live production data stream, and the historical incident repository used for RAG is still relatively small, which constrains retrieval diversity. Moreover, the quality of the final RCA report depends on both the relevance of the retrieved context and the performance of the generative model. In cases where the embedding model or language model fails, the system falls back to a basic report, highlighting the need for robust error handling and service resilience. Finally, while the pipeline automates analysis, it does not yet perform automated remediation.

13. Future Work

Future work will focus on addressing these limitations and expanding the system's capabilities. Key directions include integrating real-time streaming data sources so that evidence collection uses live metrics and logs, enlarging and continuously updating the historical incident knowledge base, and adopting hybrid retrieval methods that combine keyword search with vector similarity to improve context recall. Additional enhancements may include automated remediation or alert escalation suggestions, richer evidence sources such as pipeline logs and additional telemetry, and extension of the architecture to multiple models or broader categories of incidents.

A further extension involves collecting user feedback on generated RCA reports and using it to improve prompt design, retrieval quality, and knowledge-base curation over time.

14. Operational Impact

Beyond technical automation, the proposed RCA system has practical organizational value. It supports faster knowledge transfer, improves consistency in post-incident communication, and provides a reusable incident artifact that can assist on-call engineers, platform teams, and management stakeholders. By converting monitoring evidence into structured explanations and recommendations, the system strengthens both operational efficiency and reporting discipline in ML operations environments.

15. Conclusion

We presented an AI-driven RCA agent for MLOps monitoring that automatically investigates model performance issues and provides structured explanations and recommendations. By combining monitoring alerts, evidence collection, retrieval-based historical reasoning, and generative AI, the system transforms RCA from a manual and fragmented activity into one that is rapid, consistent, and readily actionable. This work demonstrates how modern AI and retrieval techniques can be embedded into MLOps workflows to improve reliability, transparency, and response efficiency in managing machine learning systems at scale.